```
prog_char ans01a[] PROGMEM = "  As I see it   ";
prog_char ans01b[] PROGMEM = "      yes       ";

prog_char ans02a[] PROGMEM = "     It is      ";
prog_char ans02b[] PROGMEM = "    certain     ";
.
.
.
prog_char ans20a[] PROGMEM = "      Very      ";
prog_char ans20b[] PROGMEM = "    doubtful    ";
```

You'll have to imagine the rest of the character arrays declared in a similar manner. Once this is done, we would need to create a lookup table for each one of these messages, as follows:

```
const char* allAnswers[] PROGMEM = {
  ans01a, ans01b, ans02a, ans02b, ans03a,
  ans03b, ans04a, ans04b, ans05a, ans05b,
  …
```

While we have chopped out most of the array to save paper, this part of the code looks more like the previous example array `message[]`. This array is set up as a pointer to each of the possible messages and would be accessed later in our code with the following line:

```
strcpy_P(buffer, (char*)pgm_read_word(&(allAnswers[thisAnswer])));
```

And here is more of that avr-libc voodoo that we are only going to briefly look at. The `strcpy_P()` function will copy a target string stored in program memory to a location in RAM memory. In this case we would need to declare an array named `buffer[]` with a dimension of 30 to temporarily store each individual line of text in RAM. This line then uses the `pgm_read_word()` macro to access the program memory that stores the array of messages specified in the `allAnswers[]` array at the location given by the variable `thisAnswer` to get a string of text. Dump all of that into the `buffer` array and it can be used by the Arduino functions, including `lcd.print()`, just like nothing happened. That may not make a whole lot of sense right now, but just remember that sometimes it is necessary to store large arrays of text and this is about the only way to do it without having RAM problems. There is hope because you can pretty much copy this structure verbatim and swap out the array and variable names, as well as the text strings to suit the project at hand whenever you need to.

For more on using `PROGMEM` on the Arduino, you can start with the Arduino web site at the top of the following list, followed by the avr-libc documentation and a helpful tutorial for putting this stuff in action.

- `www.arduino.cc/en/Reference/PROGMEM`
- `www.nongnu.org/avr-libc/user-manual/group__avr__pgmspace.html`
- `www.teslabs.com/openplayer/docs/docs/prognotes/Progmem%20Tutorial.pdf`

Using EEPROM

Unlike program memory, it's fairly easy to store data in the microcontroller's internal EEPROM, although we have to worry a little about the maximum 100,000 or so read/write cycles before the EEPROM stops working. EEPROM stores data in 1-byte sections with the Arduino Uno storing up to 1024 bytes. Listing 8-5 revisits that flicker sketch from earlier, but this time it puts the brightness levels into EEPROM first and reads the data from that memory instead of the array.